

TECHCATALYST DE 2026

Pandas, Polars & shipping to GCS

DataFrames for cleaning at scale, a look at Polars, and the Week 2 deliverable: S3 -> bronze -> silver.

Week 2 Day 4

DataFrame thinking

Cleaning & transforming

Joins & groupby

Pandas vs Polars

Mini-project: ship it to GCS

Today's agenda

Time	Block
8:00 to 8:25	Recap + PR review pairs
8:25 to 9:45	pandas core: read, inspect, select, clean, live coding
10:15 to 11:05	Transform: groupby, datetime, merge preview
11:05 to 11:40	pandas vs Polars: same task, different API
1:00 to 1:25	S3 -> GCS: ADC, bronze, silver, and evidence
1:25 to 3:45	Mini-project: public S3 -> GCS bronze -> GCS silver
4:25 to 5:00	Demos, recap, Week 3 preview. Enjoy the long weekend!

The DataFrame mental model

A DataFrame is a dict of columns (Series), each with one dtype, sharing an index.

Think column-wise, not row-wise: `df["tmax"].mean()` , no loops.

Vectorized operations are 10 to 100x faster than for-loops over rows , and easier to read.

Yesterday's list-of-dicts from the API? `pd.DataFrame(records)`. That's the bridge.

```
in portpandas as pd
df= pd.DataFrame(records)          # list of dicts -> table
df= pd.read_parquet("noaa_ghcnv2_2026-01.parquet") # or files
df.shape, df.dtypes                # ALWAYS look first
```

Inspect before you touch

Call	Tells you
<code>df.head()</code> / <code>df.sample(5)</code>	What the data looks like (sample > head for skew)
<code>df.shape</code>	Rows × columns
<code>df.info()</code>	Dtypes + non-null counts , your data quality x-ray
<code>df.describe()</code>	Distributions: spot the -9999 sentinel instantly
<code>df["station"].value_counts()</code>	Categorical sanity check
<code>df.isna().sum()</code>	Missing values per column

Professional habit: never transform a dataset you haven't profiled. Five calls, two minutes, every time.

Select & filter

```
df[["station", "taxi", "name"]]      # columns
df.loc[df["taxi"] > 110]              # boolean mask (rows)
df.loc[df["station"] == "M A1B0 S", "taxi"]  # rows AND columns

mask = (df["taxi"] > -40) & (df["taxi"] < 130) # combine: & | ~ ()
df.loc[~mask, "taxi"] = pd.NA         # pandas 3.0+: avoid chained assignment

df.iloc[5, 3]                        # positional when you must
```

loc = labels & masks (your default). iloc = positions. Parentheses around every condition in a combined mask.

Clean: types, missing, duplicates

```
df["tm_ax"] = pd.to_numeric(df["tm_ax"], errors="coerce") # bad -> NaN
df["date"] = pd.to_datetime(df["date"], errors="coerce")

df["snow"] = df["snow"].fillna(0.0) # impute where it MEANS zero
df = df.dropna(subset=["tm_ax", "date"]) # drop where it's fatal

df = df.drop_duplicates(subset=["station", "date"])
df = df.rename(columns=str.lower) # consistent column names
```

errors="coerce" errors="coerce" is the workhorse: invalid values become NaN you can count and decide about, instead of a crash or silent strings.

Transform: groupby & merge

```
# split-apply-combine: stats per group
by_station = (df.groupby("station")
               .agg(days=("date", "count"),
                   avg_tm_ax=("tm_ax", "mean"))
               .reset_index())

# enrich readings with station metadata
df = df.merge(station_meta, on="station", how="left")

df["month"] = df["date"].dt.month      # date time accessor
df["is_freezing"] = df["tm_in"] <= 32
```

how="left" keeps all readings even if a station is unknown. Track row counts after each merge.

Polars: the new kid worth knowing

```
in port polars as pl

df = pl.read_parquet("noaa_ghcn_d_2026-01.parquet")
result = (df.filter(pl.col("tm_ax") > -40)
          .group_by("station")
          .agg(pl.col("tm_ax").mean().alias("avg_tm_ax")))
```

API

separate from pandas, with expressions and eager or lazy execution

Expression chains read like SQL; lazy mode (`pl.scan_parquet`) optimizes the whole query before running.

When Polars: new code where speed and memory matter, and you can learn a different API.

When pandas: ecosystem glue, most tutorials, broad library compatibility. Learn pandas first, then compare.

Shipping to GCS from Python

```
from google.cloud import storage

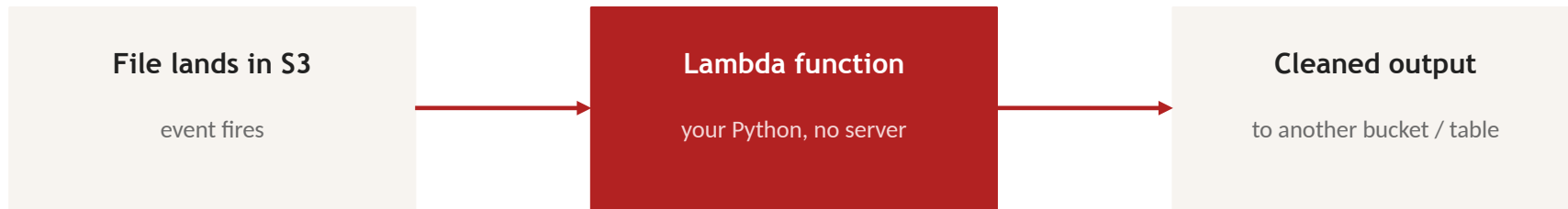
client = storage.Client() # a class! (Day 3)
bucket = client.bucket("techcatalyst-de-2026-your-use-name")

df.to_parquet("data/silver/weather/weather_clean.parquet")

blob = bucket.blob("silver/weather/weather_clean.parquet")
blob.upload_from_filename("data/silver/weather/weather_clean.parquet")
print(f"shipped:gs://{bucket.name}/{blob.name}")
```

Use bronze for raw bytes and silver for cleaned parquet. Auth uses gcloud application-default login, no key files.

AWS overlay: the serverless pattern



Event-driven: code runs only when data arrives; you pay per 100ms of execution.

GCP twin: Cloud Run functions triggered by GCS events via Eventarc.

Concept only today. Week 3 adds managed analytics and pipeline services.

Mini-project: S3 -> bronze -> silver

Goal, the Week 2 deliverable: a hand-written medallion pipeline using everything from this week.

180 min

1. Read the provided weather raw CSV locally first, then read the same source anonymously from public S3.
2. Profile, then clean in pandas: dtypes, missing policy, dedupe, lowercase columns, 2+ derived columns.
3. Upload raw bytes to GCS bronze, then upload cleaned parquet to GCS silver.
4. Add a Polars row-count comparison, docstrings, a `__main__` guard, and a README explaining run steps and decisions.
5. Submit via PR; review a teammate's PR with at least two substantive comments.

Deliverable: Merged PR: pipeline script, README, UV files, and silver object visible in your GCS bucket screenshot.

Key takeaways

- 1 Profile first (info, describe, isna). Never transform what you have not looked at.
- 2 `errors="coerce"`, `fillna` vs `dropna`, `drop_duplicates`: the cleaning core you'll use weekly.
- 3 `groupby` is split-apply-combine; `merge` is a JOIN. Week 3 makes both SQL.
- 4 Polars: same DataFrame ideas, different API, useful between pandas and Spark.
- 5 You shipped real data to the cloud by hand. Week 3: SQL and BigQuery take over from here.